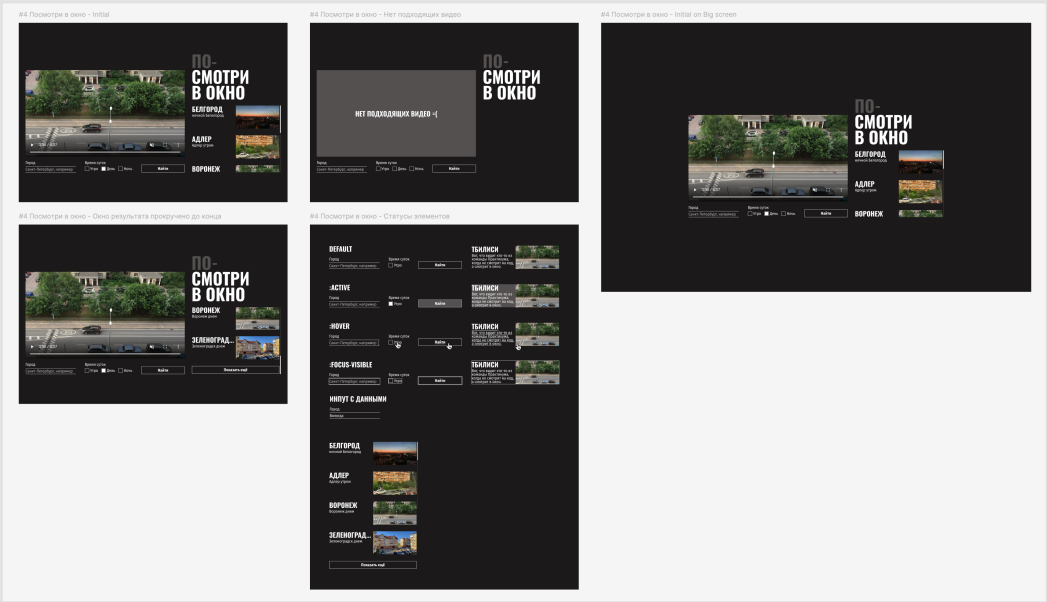
f

Проектная работа «Посмотри в окно»

В этой проектной работе вам предстоит написать CSS для уже работающего приложения «Посмотри в окно». Вы дополните файл style.css так, чтобы интерфейс соответствовал макету проекта.



В последнем уроке этой темы, «Сдача проектной работы», к вам в аккаунт GitHub автоматически скопируется стартовый репозиторий. Поэтому лучше всего двигаться в таком порядке:

1. Ознакомиться с описанием проекта в этом уроке.
2. Ознакомиться с чек-листом — в следующем.
3. Получить заготовку — в последнем.

Когда склонируете заготовку на свой компьютер, вернитесь в этот урок и приступайте к выполнению проекта.

Шаг 0. Предварительная подготовка

Изучите макет проекта «Посмотри в окно».

В стартовом репозитории вы получите готовые файлы:

- всю HTML-разметку;

- CSS для компонентов прелоадера, кастомного скроллбара и вывода ошибки;
- файлы шрифтов и стили для них;
- файл со скриптом, в котором описана логика работы приложения.

Открыв страницу из заготовки, вы увидите подгруженные видео и даже сможете выполнить поиск.

Описание функциональности приложения

- При загрузке страницы скрипт получает данные из внешнего источника, отрисовывает пять карточек с видео и кнопку, чтобы подгрузить дополнительные карточки. Ещё он подставляет адрес первого из загруженных роликов в тег <video> внутри крупного блока на странице.
- Скрипт отслеживает клик по карточкам и меняет текущее видео в зависимости от выбранной карточки.
- Также скрипт следит за отправкой формы. После отправки он ищет в базе данных совпадения по введённым параметрам и перерисовывает страницу с данными, полученными из нового запроса.
- В случае ошибок на место блока с видео подставляется блок с сообщением об ошибке. А пока идёт поиск, в блоки с видео и карточками подставляются прелоадеры, отображающие анимацию процесса загрузки.

При работе над вёрсткой вам нужно сохранить эту функциональность. Поэтому постарайтесь минимизировать свои действия с HTML-кодом.

Сейчас его достаточно, чтобы реализовать задачу. Не меняйте заданные имена классов и css-стилизацию, которая добавлена в заготовке проекта.

Устройство HTML-кода

Изучите код в файле index.html. В нём есть кое-что новое — теги <template>. Это шаблоны для разметки, которая появится на странице только по запросу. Они нужны для разработки на JavaScript и создания интерактивности. Эти теги собраны в одном месте — в конце тега <body>.

При загрузке страницы браузер не отрисовывает содержимое тегов <template>, но он знает об их существовании. В нужный момент программист может:

1. Создать сколько угодно экземпляров разметки из этого тега на JavaScript.
2. Подставить в эту разметку любые значения текста и атрибутов.
3. Вставить полученный экземпляр разметки в нужную часть страницы.

В коде четыре таких шаблона:

- Шаблон пункта списка с карточкой видео — в него подставляются данные для создания нужного количества карточек.
- Шаблон прелоадера — пока данные загружаются, для зон, где будет контент, создаются экземпляры прелоадера. Когда контент готов, прелоадеры убираются.
- Шаблон кнопки «Показать ещё» — кнопка ставится в конец списка карточек каждый раз, когда доступны дополнительные видео. Нажатие на кнопку подгружает пять

дополнительных карточек.
- Шаблон блока с ошибкой — вставляется на страницу при возникновении ошибки, текст на шаблоне зависит от типа ошибки.

Важно понимать, что стилизация экземпляров, которые созданы из шаблонов, не отличается от стилизации любых HTML-элементов. Вы всё так же можете использовать селекторы для элементов, которые описаны внутри шаблонов, и писать для них CSS-правила, — они применятся к экземплярам.
Вот пример:

```
<template class="more-button-template">
  <button class="button more-button">Показать еще</
button>
</template>
```

Для такой конструкции в HTML сработает следующий CSS:

```
.more-button {
    width: 100%;
}
```

Он применится к любой из кнопок «Показать ещё» на странице.

Шаг 1. Лейаут

В файле style.css уже есть часть базовых стилей. Они сбрасывают отступы и некоторые браузерные стили и добавляют минимальный набор стартовых правил.
Обратите внимание, что размеры сайта зафиксированы. Весь макет можно верстать в абсолютных единицах измерения.

Первым делом попробуйте собрать общий лейаут страницы. Сделайте так, чтобы взаимное расположение больших секций сайта примерно соответствовало макету.

1. Задайте флекс для контейнера .page.
2. Задайте флекс для .content и расположите дочерние элементы .result и .content__details в соответствии с макетом.

Задачи, которые вам предстоит для этого решить:

1. Выровнять блок .content по центру внутри блока .page. Сделать это надо по сетке флекс или грид как по горизонтали, так и по вертикали.
2. Расположить элементы .content в строку, прижав к низу контейнера по вертикальной оси.
3. Задать ширину .content и отступы между его элементами.
4. Расположить элементы .search-form в строку, пропорционально распределить между ними свободное пространство и прижать их к верхней части контейнера. Сделать это надо по вертикальной оси.
5. Установить жёсткую высоту для .content__list-container и добавить скролл при переполнении. Скроллбар (полоса прокрутки) скрывать не нужно, он уже стилизован в соответствии с макетом.
6. Растянуть .content__details на всю доступную ширину и высоту контейнера и добавить отступы между заголовком и зоной с карточками.

Попробуйте на этом этапе прийти примерно к такому результату:

Шаг 2. Уточнение размеров
Подгоните ряд элементов под размеры, чтобы страница стала похожа на макет. Это позволит детальнее прорабатывать компоненты уже в близком к готовности лейауте.
Выполните следующие шаги:

1. Задайте размеры контейнеру .result__video-container и добавьте внешний отступ снизу.
2. Поместите видео в этот контейнер так, чтобы оно занимало всю ширину и высоту и не выходило за пределы. Используйте object-fit.
3. Создайте отступы между элементами .content__list-item. Используйте свойство gap для .content__list.
4. Создайте отступы между кнопкой Показать ещё и последней карточкой .content__list-item. Используйте свойство gap для .content__list-container.
5. Внутри контейнера .content__video-card расположите элементы в одну строку и прижмите их к верху контейнера.
6. Задайте высоту и минимальную ширину для изображения .content__video-card-thumbnail. Используйте object-fit и object-position.
7. Сделайте так, чтобы .content__video-card-description-container занимал всё доступное пространство (не занятое картинкой). Вам поможет свойство flex-grow.
8.

9. Попробуйте добиться примерно такого результата:

Шаг 3. Стили текстов

Задайте текстовым элементам стили в соответствии с макетом. Сейчас для всего сайта заданы глобальные настройки шрифта, но для некоторых элементов они должны отличаться.

Сделайте следующее:

1. Оформите текст элементов .search-form__fieldset-title. Обратите внимание на вес шрифта, размер, отступы.
2. Оформите элемент .title и вложенный в него .content__accent.
3. Уберите дефолтное подчёркивание и замените цвет текста у ссылки .content__card-link.
4. Оформите текст .content__video-card-title и учтите возможность переполнения, заменяя излишний текст на “...”.
5. Обратите внимание на значение line-height для .content__video-card-description в макете. Здесь тоже нужно учесть возможность переполнения, для этого воспользуйтесь свойством line-clamp.

Обращайте особое внимание на интерлиньяж. Отчасти из-за необычного использования высоты линии макет приобретает задуманную дизайнером брутальность.

Попробуйте прийти к такому результату:

Шаг 4. Стилизуем формы и их состояния

Для работы с элементами формы понадобится

паттерн .visually-hidden. С его помощью вам нужно скрыть лишние лейблы и браузерные чекбоксы, чтобы потом нарисовать поверх них свои.
Добавьте в ваш код класс .visually-hidden. В HTML он уже добавлен необходимым элементам. Остались стили. Вот они:

```
.visually-hidden {
  position: absolute;
  inline-size: 1px;
  block-size: 1px;
  overflow: hidden;
  clip: rect(0 0 0 0);
  clip-path: inset(50%);
  white-space: nowrap;
}
```

Самостоятельно сверстать кастомные элементы форм — важная часть этой проектной работы.
На этом этапе не верстайте состояния инпута и чекбоксов. Отложите это до следующего шага.
Пусть чекбоксы пока не нажимаются.
Выполните следующие шаги:

1. Расположите чекбоксы в линию с помощью обёртки .search-form__checkbox-list. Используйте display: flex; и gap.
2. Сбросьте border у .search-form__fieldset.

Стилизуем обёртки для поискового инпута и чекбоксов

1. Расположите содержимое .search-form__label в строку с помощью сетки флекс.
2. Центрируйте содержимое внутри .search-form__label по вертикальной оси, используя свойство align-items.

3. Позвольте обёртке принять размер содержимого, не растягивая её на всю доступную ширину. Вам пригодится правило width: fit-content;.
4. Добавьте отступ между чекбоксом и его подписью. Удобный способ — использовать gap для .search-form__label.

Стилизуем текстовое поле ввода

При вёрстке текстового поля обращайте внимание на размеры, границу, цвет текста, семейство и размер шрифта, фон. Некоторые из этих свойств не наследуются. Стилизуйте input (текстовое поле ввода):

1. Задайте стиль текстовому полю.
2. Оставьте border сверху и снизу.
3. Установите цвет шрифта #fff для текста при вводе.
4. Установите цвет шрифта rgb(255 255 255 / 0.7) для подсказки .search-form__textfield::placeholder.
5. Установите цвет фона согласно макету.
6. Текстовому полю задайте свойство appearance: none;. Так в разных браузерах будут нужные вам стили.
7. Факультативный шаг. Когда элемент в фокусе — добавьте границу вокруг с использованием outline. У этого элемента нужно сбросить обводку в состоянии фокуса и вместо этого сделать границу у лейбла, который его окружает. Лейбл должен получать границу, только когда поле находится в состоянии :focus-visible, но не :focus. Чтобы

это реализовать, можно применить хитрую комбинацию псевдоклассов на классе лейбла — :has(:focus-visible).

Стилизуем чекбокс

1. Добавьте оформление для псевдочекбокса .search-form__pseudo-checkbox. Установите высоту и ширину и стилизуйте границу border. Назначьте внутренний отступ в 1px — он понадобится для создания отступа между границей и внутренним закрашенным элементом.
2. Добавьте псевдоэлемент ::after для класса .search-form__pseudo-checkbox. Для псевдоэлемента установите свойство content — без него псевдоэлемент отображаться не будет.
   Сделайте псевдоэлемент блочным и установите белый цвет фона.
   Задайте значения ширины и высоты, равными 100% — это обеспечит гибкость закрашенного квадрата, который будет автоматически подстраиваться под размер псевдочекбокса.
   Осталось скрыть закрашенный элемент — сделайте его полностью прозрачным, установив свойство opacity: 0.
3. Теперь стилизуем внутреннюю закрашенную часть. Задача — сделать так, чтобы при выборе чекбокса стилизованный в ::after псевдоэлемент был полностью непрозрачным. Для этого установите свойство opacity: 1. Стиль непрозрачности добавляйте в момент, когда чекбокс .search-

form__checkbox активен — за это отвечает псевдокласс :checked.
4. Декорируйте текст чекбокса — сделайте так, чтобы при наведении на чекбокс у текста появлялось подчёркивание. Обратите внимание, что подчёркивание должно появляться при наведении не только на сам текст, но и на любую точку лейбла чекбокса. При этом текст поискового инпута при наведении подчёркиваться не должен.
5. Факультативный шаг. Когда чекбокс в фокусе — добавьте границу вокруг с использованием outline. Если вы применяли псевдокласс :has() для текстового инпута, здесь тоже будет работать это правило. Вам не придётся прописывать отдельно состояние :focus и :focus-visible для чекбокса. Но если вы пошли другим путём, подумайте, как добиться того, чтобы состояние :focus не создавало обводку у родительского элемента, а состояние :focus-visible создавало.

Шаг 5. Стилизуем кнопки и их состояния

На странице две кнопки: одна в форме, другая в списке карточек. У них есть общие стили, но у кнопки в списке карточек есть и собственные. Чтобы всё сделать как надо, созданы три класса: .button, .search-form__submit-button и .more-button. Подумайте, какие стили должны быть в каждом из них.

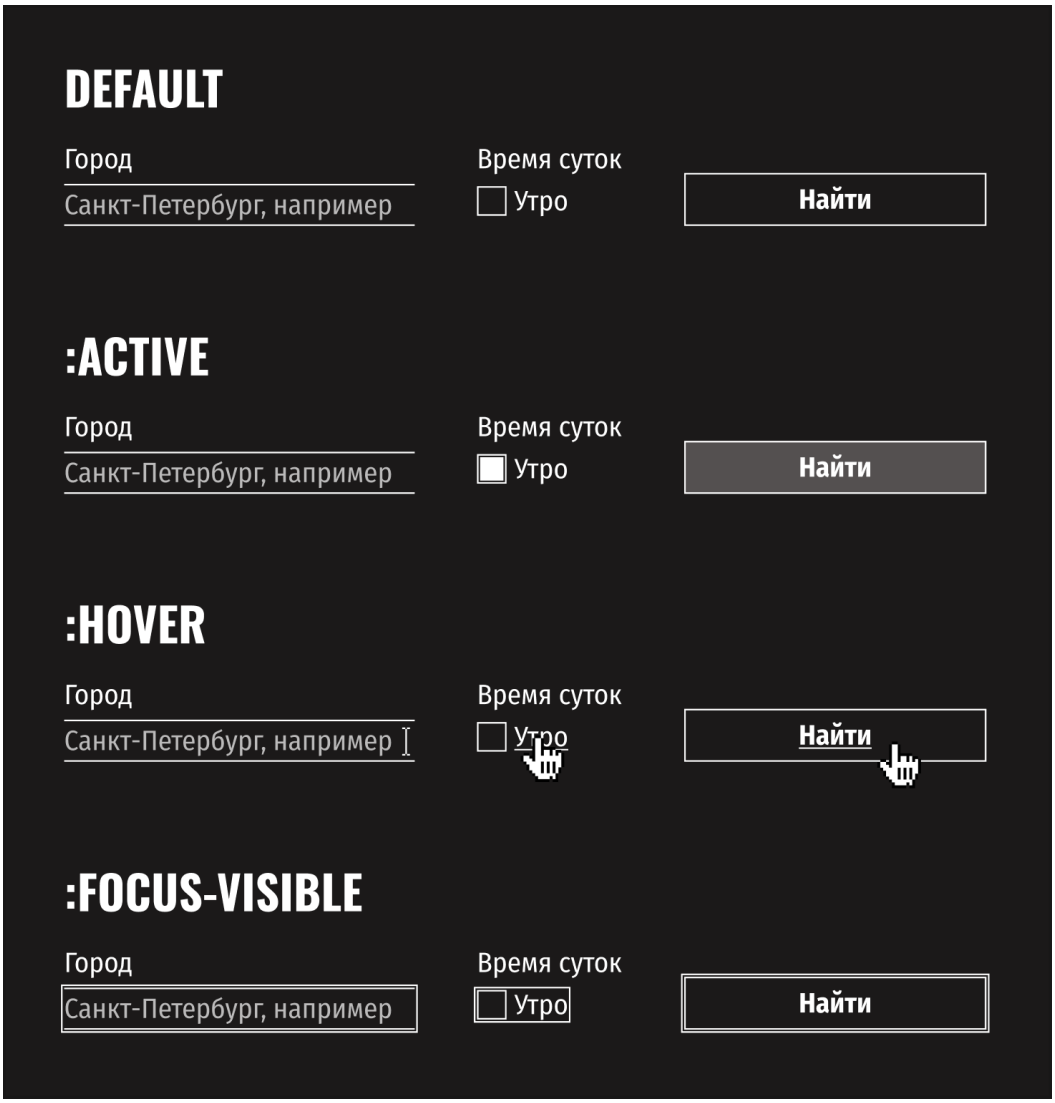
1. Для общего класса кнопки задайте границу, цвет, шрифт и прозрачный фон.
2. Пропишите индивидуальные стили для кнопок

«Найти» и «Показать ещё».
3. Для кнопки «Найти» задайте размер и положение в конце контейнера по вертикальной оси, пригодится align-self.
4. Для кнопки «Показать ещё» задайте только размер — так, чтобы она растягивалась на всю ширину контейнера.
5. Стилизуйте состояние кнопок. Ниже будет пояснение.

У кнопок на странице три состояния: :hover, :active, :focus-visible. Эти правила применяются ко всем кнопкам:

1. При наведении — текст подчёркивается.
2. Активная кнопка — задайте цвет согласно макету.
3. Факультативный шаг: при :focus стандартную обводку нужно сбросить. Для кнопки в :focus-visible появляется обводка, заданная через outline.

Вот как это будет выглядеть:



Шаг 6. Состояния карточек

Вертикальный скролл в блоке с «окнами» скрывать не нужно. Он — часть дизайна и зафиксирован в макете.

У карточек, которые выводятся справа, тоже много состояний.

Стилизация карточки

Часть стилизации вы уже добавили, осталось поработать над ссылкой карточки .content__card-link. Надо сделать так, чтобы ссылка стала блочным элементом — это позволит отображать фоновый цвет.

Текущая карточка

Одна из карточек в списке выбранная. Это поведение уже реализовано с помощью JavaScript —

при загрузке страницы скрипт добавляет класс .content__card-link_current первой карточке, и в дальнейшем этот класс переходит к той карточке, на которую кликает пользователь. В CSS нужно добавить цвет фона для селектора .content__card-link_current.

Активная карточка

Добавим стилизацию активного состояния (:active) ссылки карточки, которое возникает при удержании кнопки мыши. Для этого понадобится установить цвет фона в соответствии с макетом.

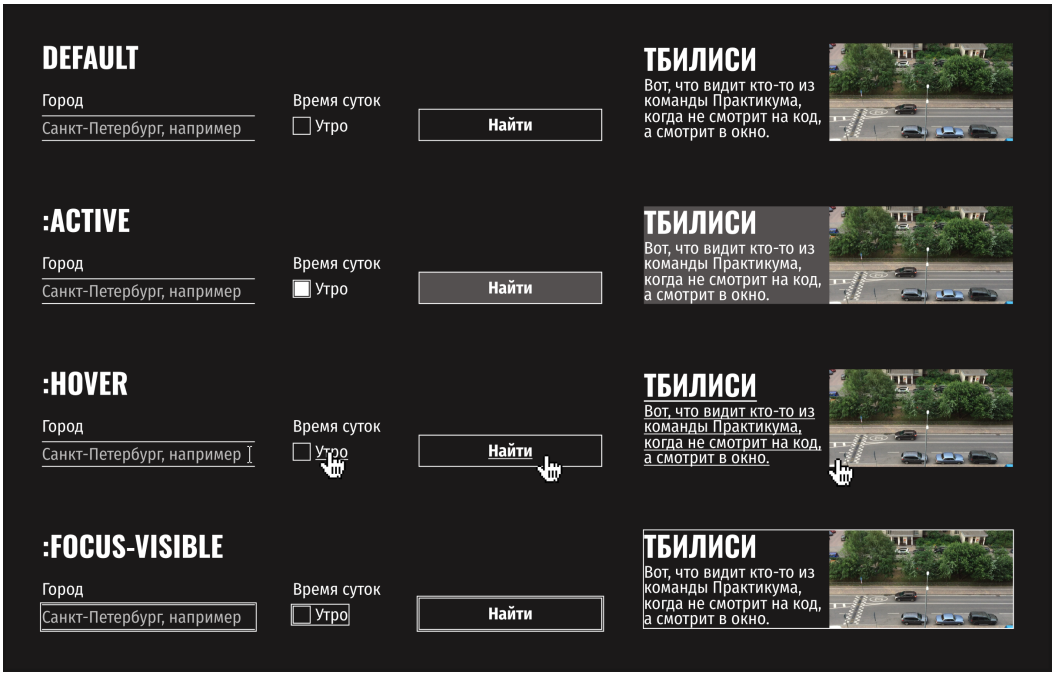
Карточка при наведении

Декорируйте текст подчёркиванием.

Карточка в фокусе

Чтобы состояние :focus-visible работало корректно, придётся сбросить стандартную обводку в состоянии :focus. При событии :focus-visible добавьте обводку с помощью outline.

Задайте padding слева и справа для .content__list-item в 3px, так обводка не прилипнет к границе карточки, если вы будете её реализовывать.



Шаг 7. Работаем с кликабельностью

Установите на кликабельные элементы cursor: pointer.

Шаг 8. Позиционирование прелоадеров

Сейчас любое действие с данными вызывает прелоадеры и они перекрывают всю страницу:

Прелоадеры должны появляться в двух местах:

- поверх .result__video-container;
- поверх .content__list-container.

Спозиционируйте эти элементы относительно. Тогда абсолютно спозиционированные прелоадеры встанут в них.

Вот результат, который вам нужен: